Tree solving and lockfiles: one answer for a whole composition

TBK SPEC 42:2026, Version 1.0

The Tebako Project

Unrestricted
November 01, 2026
TBK SPEC 42:2026, Version 1.0
Tebako
© 2026 The Tebako Project. All rights reserved

Ribose

Contents

Foreword ____ 4

1. What resolution does today, and what it cannot do ____ 4

2. The requirement closure ____ 5

3. The candidate space is the registry, as published ____ 6

4. The assignment law ____ 6

5. Conflicts are named, and they exit 81 ____ 7

6. The lockfile ____ 8

7. What this does not cover ____ 9

© 2026 The Tebako Project

All rights reserved. Unless otherwise specified, no part of this publication may be reproduced or utilized otherwise in any form or by any means, electronic or mechanical, including photocopying, or posting on the internet or an intranet, without prior written permission. Permission can be requested from the address below.

The Tebako Project

Suite 1, 8/F, New Henry House
10 Ice House Street
Central
Hong Kong

copyright@ribose.com
www.ribose.com

© 2026 The Tebako Project

Foreword

Status: PARTIAL (the payload-edge solve, the named-conflict contract and the lockfile ship with this spec; the runtime-edge seam is spec 05's resolver and is refused by name until it is wired — §2 states it).

1. What resolution does today, and what it cannot do

Resolution today answers ONE question at a time: an entrypoint declares `runtime_requirement`, the dispatcher picks the newest COMPATIBLE runtime (spec 05 §4). That is a per-edge decision, and it is correct for what it covers — a single app over a single runtime.

A composition is not one edge. A press names payloads; each payload's `requires:` names more payloads, runtimes, toolkits and data; each of those resolves against a different registry row. Today each edge is satisfied independently, in traversal order, and two edges that cannot both be satisfied are discovered as a *runtime* failure — a mount `EEXIST`, a runtime that does not match — after the package was built. First-fit over a tree is an accident that usually works.

The tree is a constraint problem, and it has one honest answer: the maximal assignment that satisfies every edge at once, or a named conflict that says which requirement, which candidates, and why.

© 2026 The Tebako Project

2. The requirement closure

The solver's input is a **requirement set**: the edges declared by the press recipe, closed over the edges of every payload it selects.

FIGURE 1

```
closure(R0):
  R = R0
  repeat
    for each r in R whose candidate is already fixed by the
assignment:
      R += the chosen payload's own requires:      (spec 03
§2.3)
  until R is stable
```

TABLE 1

edge kind	candidate space	skipped when
`language`	**spec 05's resolver, not a second copy of it**: the solver calls the dispatcher's own runtime-edge pick (engine + optional implementation + constraint against the runtime index), so a runtime edge has exactly one implementation in the ecosystem (invariant 10)	never
`runtime`	same seam, spec 30's spawned shape	never
`toolkit`, `data`	`kind`: rows with that `name`, constraint on `version`	the host triplet is absent from the edge's `triplets`: list
`app`	`kind`: app rows with that name	same as above

The runtime seam is named while it is open. The first implementation of this spec solves payload edges; a `language` or `runtime` edge is refused with `solve-runtime-edge` (exit 81) and the message points at spec 05's resolver, rather than carrying a second matcher that would drift from the dispatcher's. Closing the seam is a change to this spec's status, not a silent behaviour.

Two closure rules that are not negotiable:

- **A cycle is a named error**, not a stack overflow: an edge whose target is already on the current path fails with `solve-cycle`, naming the path as declared (it can be legitimate to *repeat* a payload in two branches — that is a DAG, not a cycle, and it resolves once).

© 2026 The Tebako Project

- **A skipped edge is recorded, not silent.** The lockfile carries `skipped: [{edge, triplet, reason}]` so a reader can see that the tree on this host is smaller than the tree in the manifest.

Where the per-payload edges come from. They are not in the registry, and that is the layer model working as designed: L3 mirrors resolution fields only, while DEPENDS is L1 (spec 03 §2.3). The caller — the press, which holds the payloads or has just read their in-image manifests — supplies each candidate's edges as a **closure map** keyed `<kind> <name>@<version>`. The solver's answer is therefore a function of three inputs: the seed edges, the closure map, and the registry documents. Nothing is fetched; a candidate whose edges are absent from the map is a named `solve-closure-missing`, never an implicit "assume no dependencies".

3. The candidate space is the registry, as published

Candidates come from the registry document(s) the press is configured with (spec 04), never from a network probe:

- `status: withdrawn` rows are **not candidates**, and a conflict whose only satisfying rows are withdrawn names them as withdrawn — "nothing satisfies" and "the only candidate was yanked" are different findings;
- a row is a candidate for a triplet only if that triplet appears in its `platforms:` (or the row is `universal`);
- the constraint language is the one already specified (spec 05 §4): ranges (`>= 3.3, < 5.0`) for pure-language edges, ABI lines (`~> 3.3.0`) for native-extension edges. The solver uses the same parser the dispatcher uses — one grammar, two callers.

4. The assignment law

An assignment is a map `(kind, name) -> (version, triplet, artifact sha256)`. Among all assignments satisfying every edge of the closure, the solver returns **the maximal one**, ordered by:

1. higher `version` is preferred for the first `(kind, name)` in canonical order (kind, then name — byte order, no locale);

© 2026 The Tebako Project

2. then higher `version` for the next entry;
3. then the lexicographically smaller artifact digest.

Consequences worth stating as laws, because they are the properties the tests must hold:

- **Determinism.** The answer is a function of (closure, registry bytes). Not of hash-map iteration order, not of traversal order, not of which mirror served the document. Two runs over the same inputs produce the same assignment and the same lockfile bytes.
- **No silent downgrade.** A lower version is chosen only when no higher one is part of any satisfying assignment, and the lockfile records the chosen version — a reader can always see what was picked.
- **One version per name.** The model has no side-by-side versions of the same payload; a requirement set that would need two versions of one name is a conflict, not a multiply-mounted tree.

5. Conflicts are named, and they exit 81

FIGURE 2

```
solve-conflict: <edge as declared>
  needs <kind> <name> <constraint> on <triplet>
  candidates: <v1>, <v2>, <v3>
  excluded: <v1> — <reason>; <v2> — <reason>
  fix: <the smallest change that would satisfy it>
```

TABLE 2

reason	meaning
`constraint`	the version exists but does not satisfy this edge's constraint
`platform`	the version exists but publishes no artifact for this triplet
`withdrawn`	the only satisfying rows are withdrawn (and this is printed)
`pinned`	another, already-required edge forces a different version of the same name

© 2026 The Tebako Project

reason	meaning
`closure`	the chosen candidate's `requires:` are absent from the closure map (`solve-closure-missing`) — the press must supply L1

Exit code 81 (`EX_TEBAKO_SOLVE`) for a conflict, 81 for lockfile drift (§6), 64 for a malformed requirement set. All three are named; none is a silent fallback. (81 is the first code above the allocated 65–80 range; the table in spec 06 gains the row in the same change.)

6. The lockfile

`tebako.lock` — YAML, one document, written by the solver and read by the press. Its bytes are a function of the assignment alone:

FIGURE 3

```yaml
lock_version: 1
assignments:                    # canonical order: kind,
then name
  - kind: data
    name: iso-codes
    version: "2024.2"
    triplet: universal
    sha256: "…"                 # the artifact's own anchor
  - kind: language
    name: ruby
    version: "4.0.7"
    triplet: aarch64-macos
    sha256: "…"
skipped:
  - edge: "toolkit gtk-layer"
    triplet: x86_64-linux-musl
    reason: triplet-not-listed
registries:                     # content, never a URL
  - snapshot: "…"               # spec 41 §2's canonical
snapshot root
```

Laws:

- **No URLs, no timestamps, no host names.** A mirror URL in a lockfile would make the same tree lock differently depending on where it was solved. The `registries:` rows carry the *snapshot root* (spec 41 §2) of the document the assignment was solved against, which is a content identity and is comparable across mirrors.

© 2026 The Tebako Project

- **Round-trip.** `solve(closure, registries)` followed by `solve(closure, registries, lock)` returns the same assignment and re-renders byte-identical bytes (`lock-roundtrip` in the test names).
- **`--locked`.** A press under `--locked` refuses to change the assignment: any difference between the lockfile and the solve is `lock-drift`, exit 81, printing the two assignments line by line. That is the flag CI uses, and the reason a green run stays green.
- **Invalidation is explicit.** A lockfile from a different `lock_version`, or one whose `registries:` snapshot no longer matches the document in hand, is refused by name — never "re-solved silently and rewritten".

7. What this does not cover

- **Inside a payload's own ecosystem.** A payload carrying `Gemfile.lock` or `package-lock.json` is authoritative for what it vendors (spec 26's press already requires a lockfile for node). The solver resolves payloads and runtimes; it does not resolve gems.
- **Signature and trust policy.** Selection happens before verification; a verified-failed artifact fails the fetch, not the solve (spec 09).
- **Deltas and transfer.** The lockfile records digests, so a fetch can plan deltas from them (spec 39) — planning is downstream of solving.
- **Cross-registry fetching.** A tree spanning two registries solves under this spec today only if both documents are configured inputs; the transport story is spec 04's.

© 2026 The Tebako Project